

Railsのメモ

Railsとは

Rubyで作成されたwebアプリ用のフレームワーク

フレームワークとは

直訳すると骨組み
様々なアプリケーションを作成するための基となる
機能を提供しており、追加の変更を行うことで、一つのシステムができる

備考

Rails以外にもJavaのspring boot
PHPのLaravel
PythonのDjango
などがある

Gemとは

rubyのパッケージのこと
rubyで利用できる様々な機能が
パッケージとして提供される

Gemのコマンド

パッケージのインストール

オプションでバージョン指定もできる

```
パッケージのインストール<hr>  
gem install パッケージ名 [-v バージョン]  
gem uninstall パッケージ名  
gem list
```

Railsの考え方

RailsはMVCモデルと呼ばれる考え方で 構成されている。

- Model ... データベースとのやり取りを行う
- View ... html作成用のテンプレート
- Controller ... 制御を行う、Modelを使ってデータ取得やViewを使ってhtmlの表示など

Railsの動作環境

Railsはサーバーで動作する
アプリケーションとなる

サーバで動作するシステムを
バックエンド

クライアントで動作するシステムを
フロントエンド

Railsの動作環境2

Railsは自機でサーバー機能を動作させ、そこにブラウザでアクセスすることで動作チェックを行う

サーバーの起動コマンド

```
rails server
```

※省略形
`rails s`

自機のサーバーにアクセス : URLとして

```
http://localhost:3000/
```

localhostは自分自身に返ってくるドメインになる。

Railsを知るうえでの補足

自分のIPアドレスを指す・・・ループバックアドレス
ドメイン名とIPアドレスを変換する・・・DNS

Railsで新規プロジェクトを作成する

新規作成

```
rails new プロジェクト名
```

Railsの基本的なアドレス

http://サーバー名/コントローラ名/アクション名/
※アクションとは
controllerに作成されたメソッドのこと

```
class ApplicationController < ActionController::Base
  # 上がコントローラ

  #下がアクション
  def hello
    render html: "hello,world"
  end
end
```

config/routes.rb

ドメイン名とコントローラーとアクションを結び付けるファイル

ここに記載されていないアクションにはアクセスできない

routes.rb

```
Rails.application.routes.draw do
  # Define your application routes per the DSL in
  https://guides.rubyonrails.org/routing.html

  # Reveal health status on /up that returns 200 if the app boots with no
  exceptions, otherwise 500.
  # Can be used by load balancers and uptime monitors to verify that the app is
  live.
  get "up" => "rails/health#show", as: :rails_health_check

  # Defines the root path route ("/")
  # root "posts#index"

  # ここが結び付けるコントローラとアクションを決めている
  root "application#hello"
end
```

ルートの設定

- ルートとは
 - サーバー名だけでアクセスできるWebページのこと
- Railsでルートを設定するときは
 - root "コントローラ名#アクション名"

とする

実行したら？

ループバックアドレスで指定ポート番号にアクセスすると実行される

scaffoldとは

直訳すると「足場」

データベースの形を設定してやると自動的に
CRUD(Create, Read, Update, Delete)の機能を作ってくれる。
ただし、余計な機能もあったりするので、あまり使用されない。

Railsのデータベース設計

Railsのテーブルは、idというフィールド名のオートナンバー型のフィールドが自動的に主キーとなる

Railsの命名規則

Railsには下記の命名規則がある。
モデル ... 英語の単数形
テーブル ... モデル名の複数形
コントローラー ... 英語の複数形（必須ではない）

Scaffoldの実行

```
rails generate scaffold モデル名 フィールド名:型 フィールド名:型
```

scaffoldの実行順番

1.プログラムの作成

```
rails new 新規プログラム名
```

2.チェンジディレクトリ

```
cd 新規プログラム名
```

3.モデルの作成

```
rails generate scaffold モデル名 フィールド名:型 フィールド名:型
```

4.データベースの作成

```
rails db:migrate
```

5.サーバーの起動

```
rails s
```

6.アクセス

※ここで注意：テーブル名は複数形であり、三単現のsがつく

```
http://localhost:3000/モデル名s/
```

テーブルの作成

モデルを作成すると、テーブルを作成する必要がある

```
rails db:migrate
```

generateの取り消し

generateで作成したファイルを削除したいときは、普通にファイルを消すのではなく、コマンドで削除する必要がある。

```
rails destroy ~
```

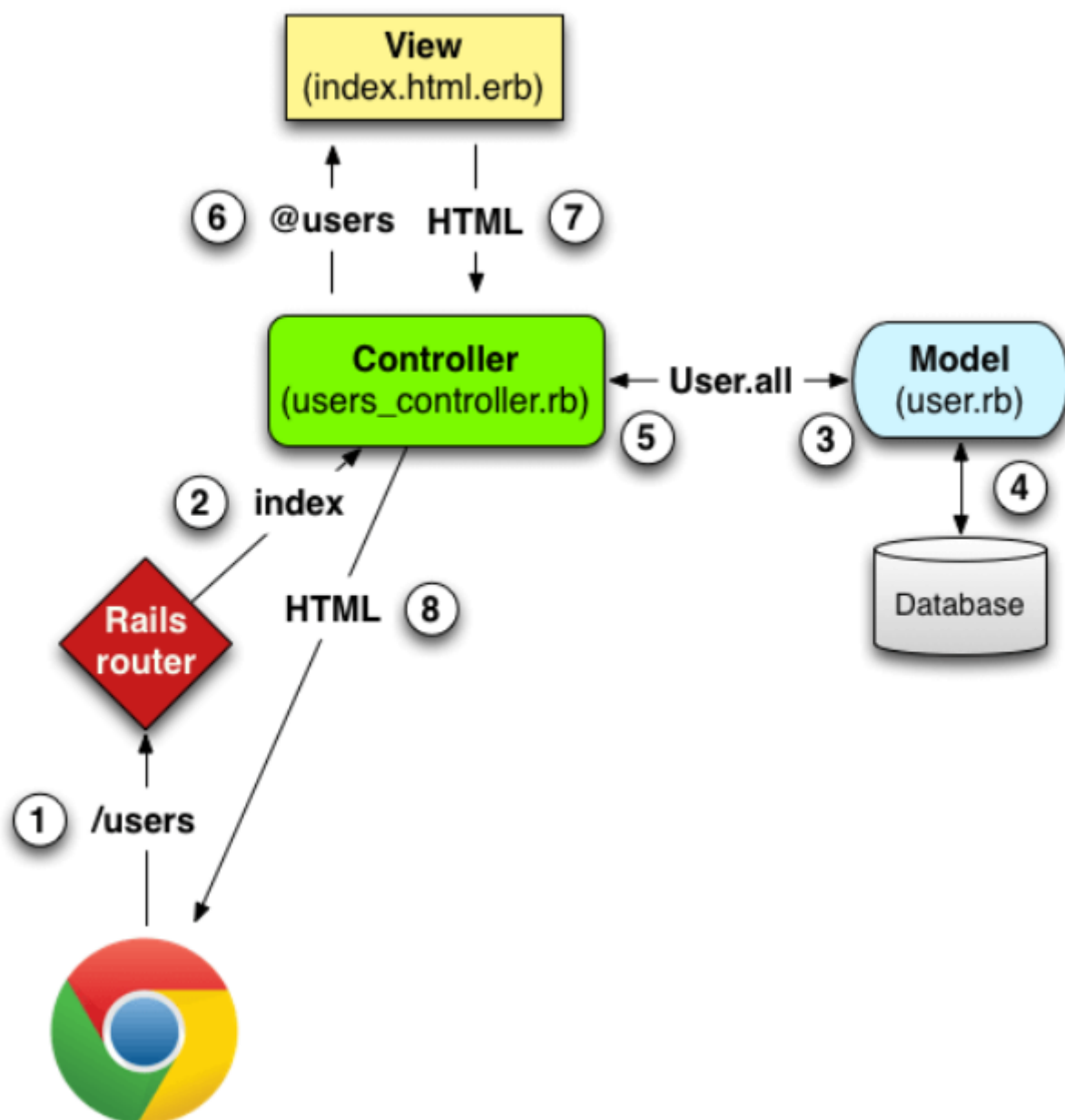
※

どのファイルにアクセスするかの設定がされているファイル
= config/routes.rb

generateで作成したファイル削除

rails destroy モデル名 フィールド名:型 フィールド名:型

MVCの挙動について



Viewファイルについて

Viewファイルはhtmlの中にRubyを埋め込んで作成される。

ソース(app/views/users)
拡張子は、html.erb(エンベデッドシステム：埋め込みプログラムのことをいう)
通常のhtmlとRuby用の<% rubyのコード %>のタグを使用する

<% ~ %> ... Rubyのコードを実行し、その結果を出力しない

<%= ~ %> ... Rubyのコードを実行し、その結果を出力する

```
#<%=から%>で挟まれているものがエンベデッド(スクリプト)である
<p style="color: green"><%= notice %></p>

<h1>ユーザー一覧</h1>

<div id="users">
  <% @users.each do |user| %>
    <%= render user %>
  <p>
    <%= link_to "Show this user", user %>
  </p>
<% end %>
</div>

#ここは<%= %>なので出力する
<%= link_to "New user", new_user_path %>
```

上記のコードの解説

render ... 出力
link_to ... aタグを作成する

@users.each do |user| でひとつずつレコードデータを取ってきている

アソシエーションの設定

テーブルの関連付けのことをアソシエーションという

Railsでテーブルを関連付けたいときは、以下の手順が必要となる

1. 1対多の多側のテーブルのフィールドにモデル名_idのフィールドをInteger型で作成する。これにより、1側のidと対応させることができる
2. 1側のモデルに

has_many : 多側のモデル名

を記述する

3. 多側のモデルに

belongs_to : 1側のモデル名

を記述をする

これにより、1側のインスタンス.多側のモデル名.フィールド名や1側インスタンス.1側のモデル名.フィールド名のアクセスが可能になる

練習問題

Q.Sales_appというプロジェクトを作成し、下記テーブルをスキュフォールドで作成する

```
モデル名 : item
フィールド名 : name...String
              suryo...integer
```

Saleの詳細表示

商品名 :

単価 :

数量 :

金額 :

が出るようにする

1. プロジェクトの作成

```
rails new sales_app
```

2. プロジェクトのフォルダに移動

```
cd sales_app
```

3. スキュフォールドの実行

```
rails generate scaffold item name:string price:integer
rails generate scaffold sale item_id:integer suryo:integer
```

4. テーブルの作成


```
rails db:migrate
```

5. アソシエーションに設定

```
app\models\item.rbに  
has_many:sale  
app\models\sale.rbに  
belongs_to:item  
を追加
```

6. 詳細表示の修正

```
item.sale.フィールド名で関連付けされたフィールドにアクセスできるので、それを使って、  
app\views\sales\_sale.html.erbを修正
```

destroyについて

```
テーブルを作成した後に、destroyを行う場合は先に  
rails db:migrate version=0  
を行う必要がある。  
ただし、いまままで登録してきたデータがすべて削除されるので注意すること  
誤ってrails destroyを先にした場合は、  
rails db:migrate version=0  
を行った後  
storage\development.splite3（ファイルのパス名）  
を削除する
```

ここままで気になっていることを補足

ホーム画面が 作成したテーブルの名前が上に表示されている画面になる

collection_select

テーブル情報をもとに、セレクトボックスを作成するもの

```
collection_select :フィールド名, モデル名.all, 取得したデータの登録をするフィールド名, :取得し  
たデータの表示をするフィールド名
```

今回のコード

```
「<%= form.collection_select :user_id ,User.all ,:id, :name%>」
```

<<解説>>

form -> ブロック変数

:user_id -> フォームが送信されたときのフィールド名

User.all -> モデル名のデータベースの全てのレコード取得

:id -> <option value="1">みたいにhtmlでいう、そのセレクトボックスのid番号になる

:name -> 選択肢の表示させるテキストになる

scaffoldでできるもの

- モデル・・・指定したモデル名のモデル
- マイグレーションファイル・・・テーブル作成用のファイル。rails db:migrateを実行することでテーブルが作成される。
- コントローラー・・・モデル名を複数形にしたファイル。トップページ用のindex、新規作成画面表示用のnew、データ登録用のcreate等が作成される。アクションと同じ名前のviewファイルが自動的に表示される。
- ビュー・・・
 - index.html.erb トップページ用
 - show.html.erb 詳細表示用
 - new.html.erb 新規作成用
 - edit.html.erb 更新用
 - _form.html.erb データ登録用（new.html.erbやedit.html.erbから呼び出し使用する）
 - _モデル名.html.erb データ表示用（index.html.erbやshow.htmlから呼び出し使用する）
- テスト用ファイル

これ以外にconfig/routes.rbにアクセス用の登録が追記される

モデルのメソッド

- .all・・・全件のデータ取得。一軒のデータを持っているインスタンスの配列になる。
- .find(id番号)・・・指定されたIDのインスタンスを取得する

railsコマンドの省略形

- rails server ... rails s
- rails generate ... rails g
- rails destroy ... rails d
- rails console ... rails c
- rails test ... rails t -> テストトリブンというものがある

コントローラーの作成

```
rails g controller コントローラー名 [アクション名 ...]
```

例: rails g controller Static Pages home help

(大文字区切りで単語を命名) パスカルケース: コントローラの名前、クラス名
(アンダーバーを付ける命名) スネークケース: ファイル名

コントローラを作成すると同時にapp/viewsにコントローラと同じ名前のフォルダが作成される
アクション名を名づけることで、コントローラにアクションが追加されまた同名のviewファイルが作成され、routes.rbにも追加される。

Railsの豆知識

- .saveは成功したときに真になる
- renderはレンダリングのこと。個人的にはrequireに少し似ている
- render "form"は_form.html.erbを呼び出しているということ
- 基本controllersで作動しており、モデル名_controllers.erbのアクションメソッドが起動する。その際に、before_action :set_〇〇とあるが、これはアクションが実行される前に自動で設定されるもの

モデル名の複数形を知りたい場合、controllersのコントローラ名がモデル名の複数形になる もしくはターミナルで"person".pluralizeのように"単数形".pluralize

controllersの挙動

```
class UsersController < ApplicationController
  # ここでset_userメソッドを起動する。
  # only: %i[ ]は括弧にかこまれたメソッドにそのset_userの前処理を適用する
  before_action :set_user, only: %i[ show edit update destroy ]

  # GET /users or /users.json
  def index #->indexメソッドが起動すれば、自動的にviewsのindex.html.erbが起動される
    @users = User.all
  end
end
```

```
# GET /users/1 or /users/1.json
def show
end

# GET /users/new
def new
  @user = User.new
end

# GET /users/1/edit
def edit
end

# POST /users or /users.json
def create
  @user = User.new(user_params)

  respond_to do |format|
    if @user.save
      format.html { redirect_to @user, notice: "User was successfully created." }
      format.json { render :show, status: :created, location: @user }
    else
      format.html { render :new, status: :unprocessable_entity }
      format.json { render json: @user.errors, status: :unprocessable_entity }
    end
  end
end

# PATCH/PUT /users/1 or /users/1.json
def update
  respond_to do |format|
    if @user.update(user_params)
      format.html { redirect_to @user, notice: "User was successfully updated." }
      format.json { render :show, status: :ok, location: @user }
    else
      format.html { render :edit, status: :unprocessable_entity }
      format.json { render json: @user.errors, status: :unprocessable_entity }
    end
  end
end

# DELETE /users/1 or /users/1.json
def destroy
  @user.destroy!

  respond_to do |format|
    format.html { redirect_to users_path, status: :see_other, notice: "User was successfully destroyed." }
    format.json { head :no_content }
  end
end
```

```
private
  # Use callbacks to share common setup or constraints between actions.
  def set_user
    # ここで@userにテーブルデータをidで指定して持ってきて@userに格納
    @user = User.find(params[:id])
  end

  # Only allow a list of trusted parameters through.
  # ここではparams.require(持ってくるシンボラ配列のキー).permit(通過してもよいハッシュキー)でユーザ
  # に変なデータを読み取らせないような工夫をされている
  def user_params
    params.require(:user).permit(:name, :email)
  end
end
```

新しいページを作成する方法

1. コントローラに外套のアクションを追加する
2. config/routes.rbにルーティング情報追加
3. views/コントローラ名の下に該当のファイルを作成

テストについて

テストをもとにシステムを作成することをテストトリブンと言う。テスト用のファイルはコントローラの作成時に自動で作成される。

※テスト用のファイルは親ディレクトリの直下のtestディレクトリ内に存在する

テストの実行方法

テストの実行方法は下記をターミナルで入力すると実行される

```
rails test
```

テストの見方

- runs → 実行されたテストの数
- assertions → アサーション（期待値と実際の値を比較してチェックする処理）の回数
- failures → 期待通りの結果にならなかったテストの数
- errors → コード自体に致命的なエラーがある数
- skips → スキップされたテストの数

- Expected: 実際になっているエラーデータ
- Actual: 本来はこうなるべきデータ

viewsの<!DOCTYPE>などのヘッダー、フッターはどこに行ったのか？ -> テストの見方

views/layoutsに存在する、application.html.erbにある

テストの見方

```
test "should get about" do #aboutというコントローラを実行し、以下のテストをしますよ
  get static_pages_about_url #static_pagesのaboutのurlとその情報を取得(Getだから)
  assert_response :success #ここでassert_responseというテストをして成功すれば、successを返す
  assert_select "title", "About | Ruby on Rails Tutorial Sample App" #titleというタグ
  # 次メッセージを確認する
end
```

Viewファイルに情報を渡す

yieldというメソッドを使うことによって変数やファイルにviewファイルからデータを渡すことができる。

```
データの設定
provide(:データ名, 値) #参照できるように設定する。キーと参照させる値
データの参照
yield(:データ名) #データ名を参照する
```

例

```
# 別ファイル
<% provide(:title, "About") %>
# htmlファイル
<title><%= yield(:title) %> | Ruby on Rails Tutorial Sample App</title>
```

注意点

もしも<%= yield %>だけだと、ビューの内容が送られる

ルーティングについて

routes.rbの主な設定は以下の通り

- get "コントローラ名/アクション名" <- 指定したURLにGET送信を許可する

- `post "コントローラ名/アクション名" <-` 指定したURLにPOST送信を許可する
- `root "コントローラ名#アクション名" <-` ルートにアクセスしたとき、指定したコントローラにアクションを実行する

例 : `localhost:3000/`だけで指定したhomeやhelpなどのアクションページに移動できる

- `"URL", to: "コントローラ名#アクション名" <-` 指定したURLにアクセスしたとき、指定したコントローラのアクションを実行する
- `resources :コントローラ名 <-` 指定されたコントローラに対し、indexやupdateなど、定型なアクションを許可する (RESTアーキテクチャー)

ヘルパーについて

ヘルパー...

各コントローラに対し一つ作成されるモジュール。コントローラとビューで利用できる。Railsが準備したシステムヘルパーと自分で作成するカスタムヘルパーがある。

moduleタグについて

様々なところから呼び出される物はmoduleタグが使われる

`app/helpers/application.html.erb`がヘルパー記述欄

```
# application.html.erbに以下
module ApplicationHelper
  # viewsでfull_titleメソッドを実行できる
  def full_title(page_title = "")
    base_title = "Ruby on Rails Tutorial Sample App"
    if page_title.empty?
      base_title
    else
      "#{page_title} | #{base_title}"
    end
  end
end
```

rails consoleについて

railsの環境でコマンド操作が行えるもの

modelやhelperなどのチェックも行える

※rails cでインタプリタが使える

```
>a= 1
>b = 2
>a+b
=> 3
というように出る。

>User.all
とすれば、全件出力
>User.find(1)
とすれば、idが1の人を出力
```

モックアップ

画面を作る設計図のこと

link_toメソッド

viewの中でアンカータグ（リンクタグ）を作成するためのメソッド。link_to表示する文字列、リンク先の文字列

※(#)シャープをリンク先のところに置く理由。リンク先が確定しない場合、シャープを置く。つまりシャープはスタブということになる

image_tagメソッド

イメージファイルを表示するためのタグ。イメージはappの中のassetsの中のimagesに保存する

```
image_tag(ファイル名, [alt: 画像が表示されなかったときに表示する文字列, オプション])
```

gemファイルについて

railsやrubyで機能を追加したいときはgemというパッケージ毎にインストールを行う。

railsの場合、プロジェクトフォルダの直下にGemfileとGemfile.lockというファイルがある。

gemのインストールについて

```
gem install gemファイル名 [オプション]
または
bundle install
```

ファイル名を指定しない場合、Gemfile.lockを参照し、該当のgemがない時はインストールする。そのあとGemfileを参照し、インストールなどを行う。

インストール後、新規でインストールされたgemはgemfile.lockに登録される。

つまりGemfileが変われば、Gemfile.lockも変わる

ブートストラップについて

レンダリング処理の時にブートストラップが使われる

パーシャルについて

railsでホームページを作成する際に様々な場所から呼び出されるものは、別ファイルにして置き、それを読み込むことで使用する。このことをパーシャルという

パーシャルの読み込み方 -> 読み込む方

`render "viewsの中のフォルダ/ファイル名"`

パーシャルの読み込み方 -> 読み込まれる方

`viewのフォルダ名/_ファイル名.拡張子`

※アンダーバーがあるファイルは基本的は取り込まれるべきファイルと考えるとよい

assetsについて

共通でつかうものやimagesなど、コンピュータ資源などが入っている

アセットパイプラインとは

javascriptやスタイルシートなどの静的なファイルを管理するもの

※一般的には資産という意味で、ITの世界では素材の意味でつかわれている

アセットパイプラインのフォルダ

- `app/assets` ... 現在のプロジェクトのアセット
- `lib/assets` ... 共有されいてるアセット
- `vender/assets` ... サードパーティ（開発支援で使う他社のパッケージ）用

config/manifest.jsについて

アセットパイプラインが働いて最後に一つにまとめられて使用される特徴がある

以下：manifest.js

```
// = link_tree ../images
// = link_directory ../stylesheets .css
// = link_tree ../../javascript .js
// = link_tree ../../../../vendor/javascript .js
// = link stimulus-loading.js
```

マニフェストファイル

assetsでディレクトリに保存されたファイルをどのように統合するかを決めるファイル。

通常はマニフェストファイルに読み込む順番を設定していく

```
スタイルシート用
  assets/stylesheets/application.css
javascript用
  assets/javascripts/application.js
```

マニフェストファイルの設定

コメントの中にどのファイルを読み込むか設定を行う

- *= require ファイル名 ... 指定したファイルを読み込む
- *= require_tree パス ... 指定したパスからツリー構造で読み込む
- *= require_self ... 自分自身を読み込む

最初にrequire_tree .をするのはなぜ

```
*= require_tree .
*= require_self
```

上のようにrequire_tree .とは、最初に自分自身を読み込むということ

※先に自分自身を読み込めば、自分自身が先に優先されるように設定される

プリプロセッサ

ファイルの結合やコンパイルなどを行うもの。

Railsの場合、スタイルシートも独特な記載方法（scss, sass）があり、それをコンパイルし、一つのファイルにまとめる

sassとは

スタイルシートに変数や外部ファイルの読み込みなどプログラムの要素を取り入れたもの。サスまたはサースと呼ばれる。

box1クラスの中のbox2に対する設定↓↓↓

```
/*cssの場合*/
.box{
  設定;
}
.box h1{
  設定;
}
/*sassの場合*/
.box{
  設定;

  h1{
    設定;
  }
}
```

```
/*例、box1とbox2のクラスの背景色を#123456にする*/
/*css*/
.box{
  background-color: #123456;
}
.box2{
  background-color: #123456;
}
/*sass*/
$変数名: 設定する物;
.box{
  background-color: $変数名;
},
.box2{
  background-color: $変数名;
}
```

a:hoverを書くには & を使う

```
a {
  color: #555;
  a {
    color: #555;
    &:hover{
      color:#444;
    }
  }
}
```

testについて 2

failuresとerrorsの違い

- failures → 期待通りの結果にならなかったテストの数
- errors → コード自体に致命的なエラーがある数

>> 本来はこのような意味合いがある

- failures -> アクセスはできたが、コードが違う
- errors -> そもそもアクセスできなかった

どんな動作でエラー、フェイラーが出るのか

エラー

テストの実行中に予期しない例外が発生した場合

- データベース接続の問題
 - 未定義のメソッド呼び出し時
- >> テスト自体が正常に完了しなかった場合

フェイラー

期待した結果と実際の結果が一致しなかった場合

- 必要なgemがインストールされていない
-

特定のURLにアクセスしたらコントローラを実行

routes.rbに以下の設定をすると可能になる

```
# localhost:3000/helpとすると、static_pagesコントローラのhelpを実行し、helpにアクセスされる
get '/help', to: "static_pages#help"
```

testのgetについて

getはそのURLにアクセスしますよという意味。

```
test "should get help" do
  get static_pages_help_url
  assert_response :success
  assert_select "title", "Help | #{@base_title}"
end
```

routes.rbの設定

特定のURLにアクセスされてら、コントローラーのアクションを実行する

```
get "特定のURL", to: "コントローラー名#アクション名"
```

統合テスト

各ページごとに行うテストを単体テスト、複数ページにまたがって行うテストを統合テストという

統合テストを行うときは、統合テスト用のファイルを作成する

```
rails generate integration_test ファイル名
# /test/integration/の中に指定したファイルが作成される
```

統合テストの主なassert

- 指定したViewファイルを利用したページか？

```
assert_template "viewファイルのパス"
```

- 指定したタグが存在しているか？

注意点：タグの中に?を使い、「値」を使うと?に代入される。また「count: 整数」とすることで、個数が正しいかチェックできる

```
assert_select "タグ"
```

test/integration

```
class SiteLayoutTest < ActionDispatch::IntegrationTest
  test "layout links" do
    get root_path
    assert_template "static_pages/home"
    assert_select "a[href=?]", root_path, count: 2
  end
end

# assert_select "a[href=?]", root_path, count: 2は、href=?の?の中に、root_pathが代入され、
# count: 2は、そのa[href=#{root_path}]が二個static_pages/homeにあるか確認する
```

統合テストの実行

統合テストを行うときは以下のコマンドを実行する

```
rails test:integration
```

なぜassertionsをしたのに、assertionsが+1ならないのか

assertionは、実行した際にチェックが行われずに終了した場合は、assertionsが+1とならず次へ行ってしまう。

OO_pathで飛ぶようにする設定

```
<%= link_to "Sign up now!", signup_path, class: "btn btn-lg btn-primary" %>

# 上のsignup_pathにパス先のヘルパーが入る
```

モデルの作成

※モデルを作成したら必ずマイグレーションを行う

```
rails generate model モデル名 フィールド名:型名 フィールド名:型名 ...
```

スキファールドと何が違うのか

スキファールドはその詳細まで作ってくれるが、モデルはしてくれない

rails consoleでデータを追加する

Userモデルにデータ追加

```
rails c
```

```
user = User.new(name: "sakane", email: "sakane@icc.core.ac.jp")

# userで入力したデータをそのままモデルに登録
user.save
```

モデルの使い方 1

- 全件のデータを取得する

```
モデル名.all
```

- 新しいインスタンスを作成する

```
モデル名.new
```

- 新しいデータを登録する

```
モデル名.create(フィールド名:値, フィールド名:値 ...)
```

```
# モデル名に直接入力するので、saveはいらない
```

インスタンスの使い方

- データを参照する

```
インスタンス名.フィールド名
```

- データを登録する

```
インスタンス名.フィールド名 = 値
```

- インスタンスをデータベースに保存

```
インスタンス名.save
```

データ取得

該当のIDのデータを取得する

```
モデル名.find(id番号)
```

該当のデータを削除

```
インスタンス.destroy
```

ID以外で検索する

```
モデル名.find_by(フィールド名: 値)
```

最初のデータを取得する

```
モデル名.first
```

最後のデータを取得する

```
モデル名.last
```

バリデーション

データの登録・更新時に行うチェックのこと。モデルに記述を行う

- 必須入力チェック

```
# app/models/user.rbに記述
# これはフィールド名(例: name)が何か書いてないとfalseでエラーになってしまうという設定を作る
validates :フィールド名, presence: true
```

- 長さチェック

```
validates :フィールド名, length: {長さの指定}
# 長さの指定
minimum: 数字 #... 何文字以上
maximum: 数字 #... 何文字以下
in: 範囲      #... 文字数が範囲内
is: 文字数    #... 文字数が指定された文字数
```

- 文字数のパターンチェック

```
validates :フィールド名, format: {with: 正規表現}
```

- 一意性(一つ以上は存在していないという性質)

```
# uniquenessは大文字、小文字もしっかり区別してくれる
validates :フィールド名, uniqueness: true
```



```
# 大文字・小文字を区別しない場合でバリデーションチェック -> case_sensitiveが大文字小文字区別オプション
validates :フィールド名, uniqueness: { case_sensitive: false }
```

- 数値チェック

```
validates :フィールド名, numericality: { チェック項目 }
# チェック項目
only_integer: true           #...整数型か
greater_than: 数値           #...数値より大きい
greater_than_or_equal_to: 数値 #...数値 ... 数値以上
equal_to: 数値               #...数値と同じ
less_than: 数値              #...数値より小さい
less_than_equal_to: 数値     #...数値以下
```

バリデーションの有効性のチェック

- 有効性のチェック

```
# 上記の付与した設定ルールを守っているかというのをrails consoleで確認できる。コードにも記述可能
インスタンス.valid? ... 登録できるデータならtrue, それ以外はfalseが返される
```

- テスト用のデータベースの作成

```
rails db:migrate RAILS_ENV=test
```

- rails test:modelsの見方

```
class UserTest < ActiveSupport::TestCase
def setup
  @user = User.new(
    name: "example User",
    email: "user@example.com"
  )
end

test "should be valid" do
  assert @user.valid? # @userに値が存在するか?
end

test "name should be present" do
  @user.name = " "
```

```
    assert_not @user.valid? # assert_notは失敗するか？ということで、@user.valid?で@userに  
    値が入っていないくて、失敗するのでassert(成功)ということになる  
  end  
end
```

dupとは

rails testのdupとはなにか

dupは浅いコピーであり、中身のデータをコピーせず、設計図だけコピーする感じ

```
test "email addresses should be unique" do  
  duplicate_user = @user.dup  
  @user.save  
  assert_not duplicate_user.valid?  
end
```

ダブルクリックの力

ダブルクリックをするとエラーチェックを通してしまうことがある。その対策のためのコマンドが以下だ

テーブルの変更

railsの場合、フィールド名の名前の変更や、フィールドの追加など、テーブルに変更を加えるときはマイグレーションファイルを作成し、変更内容を記述し、マイグレートを行う必要がある。

マイグレーションファイルの作成

```
rails g migration ファイル名
```

マイグレーションファイルはmigrateの中に追加する

※マイグレーションファイルのファイル名は規約によってつける

```
作業_作業内容_to_テーブル名_フィールド
```

例文

```
# usersテーブルのemailフィールドにindexを追加するというコマンド  
rails g migration add_index_to_users_email
```

練習問題

books_appというプロジェクトを作成しなさい

- 1.下記コントローラを作成

authorsコントローラ

- 2.下記ビューを作成

menu

- 3.menu.html.erbを下記のように変更

※著者一覧に対しauthorsコントローラのindexに対しリンクを作成

メニュー
著者一覧
本一覧

- 4.下記ビューを作成

index

- 5.index.html.erbを下記のように変更

著者一覧

- 6.3と5にそれぞれアクセスできるようにする
- 7.以下のテストを作成
 - ルートにアクセスしたら、menuが開く
 - menuの中に著者一覧へのリンクがある
 - 5にアクセスできる
- 8.下記モデルを作成

- モデル名 : **book**
 - フィールド : **name string** 必須入力 20字以内
 - フィールド : **price integer** 整数値チェック
- 9.8に対するテストを作成

セキュアなパスワードを追加する

パスワードなどの機密データは暗号化して保存するのが、一般的になる。この暗号化のことを**ハッシュ化**という。

パスワードの保存の手順

- 1.パスワードのフィールドを作成する
- 2.仮想フィールドの作成をする
- 3.passwordとpassword_confirmationが同じだったらパスワードのフィールドに保存する

※新規登録時、二回ほど認証してやるやつ、password_confirmationに二回目を入力して、passwordとpassword_confirmationをぶつけて一緒だったら、パスワードを登録できるという仕組み

```
# パスワード用のフィールドを、追加する。
rails g migration add_password_digest_to_users password_digest:string
```

パスワードの作成

1.モデルに

```
has_secure_password
```

を記述。仮想的なフィールドのpasswordとpassword_confirmationが作成される。

2.gemにbcryptを追加

Gemfile

```
gem "bcrypt", "3.1.18"
```

ハッシュ化用gem

```
rails db:migrate version=0 RAILS_ENV=test
rails db:migrate RAILS_ENV=test
```

Railsのデバッグ

railsのデバッグ方法として二つのやり方がある。

- 1.debugメソッドを使う

Viewやコントローラーなどで
debug (変数名)
とする

app/views/application.html.erb

```
<body>
  <%= render "layouts/header" %>
  <div class="container">
    <%= yield %>
    <%= render "layouts/footer" %>
    <%= debug(params) if Rails.env.development? %> <- 開発が動いていたら、debug
を出力しますよ。
  </div>
</body>
```

- 2.pメソッドを使い、コンソールに出力

viewやコントローラーなどで
p 変数名
とする
コンソール上に出力する

controller/static_pages_controller.rb

```
# pメソッドはデバッグ用に使われる出力。ここで書いた内容はコンソールに出力される。
def about
  p "-----"
  p "This is the about page."
  p "-----"
end
```

- 3.debuggerメソッドを使う

```
viewやコントローラなどで
  debugger
とする。
デバッグ用のコンソールが立ち上がり、そこで変数などを確認できる。
rails sをしたコンソールに(rdbg)と表示され、そこで@user.nameとかと書くと、データが表示される。
Ctrl + d で抜けられるが、サーバーが止まる。
```

routes.rbのresourcesメソッドについて

routes.rbに

```
resources :コントローラ名
```

とすることで、以下のルーティングが設定される（get "/○○"の設定がいっぱいされるということ）

- /コントローラ名 ... indexアクションを呼び出す。一覧用
- /コントローラ名/ID番号 ... showアクションを呼び出す。詳細表示用
- /コントローラ名/new ... newアクションを呼び出す。新規登録用
- /コントローラ名/ID番号/edit ... editアクションを呼び出す。修正用
- post/コントローラ名 ... createアクションを呼び出す。登録処理
- patch /コントローラ名/ID番号 ... updateアクションを呼び出す。更新処理
- delete /コントローラ名/ID番号 ... destroyアクションを呼び出す。削除処理

※resourcesを付けるうえで必要なこと

コントローラとビューにshow設定が必要になる（コントローラでshowを設定して、show.html.erbを実行する）

/controller/users_controller.rb

```
class UsersController < ApplicationController
  def new
  end

  def show
    @user = User.find(params[:id]) #paramsはリクエストパラメータからidをもらってくる処理。URLで指定するときに、
  end
end
```

form_withについて

railsでフォームを作成するときはform_withメソッドを使う

```
form_with(model: インスタンス名) do |変数|
  変数.フォームコントローラ名(:インスタンスのプロパティ[, オプション])

  変数.submit "ボタンに表示するテキスト" #submitを押せば、controllerのcreateメソッドが起動する
end
```

アクセス方法

フォームに入力された値は
params[モデル名][:インスタンスのプロパティ]
でアクセスできる

主なフォームコントローラ名

.label :プロパティ[表示される文字列] ... ラベルの表示

.text_field :プロパティ[オプション] ... テキストフィールドの表示

.text_area :プロパティ[オプション] ... テキストエリアの表示

.select :プロパティ, 表示する内容の配列 ... セレクトボックスの表示

.submit :プロパティ, 表示する文字列 ... 送信ボタンの表示

.button :プロパティ, 表示する内容の配列 ... ボタンの表示

check_box :プロパティ ... チェックボックスの表示

radio_button :プロパティ, プロパティに入れる値 ... ラジオボタンの表示

collection_select :プロパティ, 配列, プロパティに入れる値, 表示する内容 ... コレクションセレクトの表示

コード例

```
<div class="row">
  <div class="col-md-6 colmd-offset-3">
    <%= form_with(model: @user) do |f| %>
      <%= f.label :name, "名前" %> # :nameとは何なのか？これは、<label></label>で
      囲んだ時、ラベルを押したらテキストフィールドにカーソルが飛ぶという設定を依然したと思う。そのこと
      <%= f.text_field :name %>
    </div>
  </div>
</div>
```

```

        <%= f.submit "データ送信" %>
      <% end %>
    </div>
  </div>

```

Rails豆知識

- タグは斜めにする
- get送信はURL事態に情報が埋め込まれている
- routes.rbにget 'static_pages(アプリの名前)/about(コントローラ名)'を作ると、自動的にstatic_pages_about_urlのような変数が自動的に作成される
- resources :authors(コントローラ名)としたらcreateやupdateなど自動的に作られる。
- gravatarについて

```

# gravatarとは、メールアドレスをハッシュ化した値を使って、グローバルに使えるアイコン画像を提供してくれるサービス。
# <show.html.erb>
<h1><%= gravatar_for @user %></h1>

# </app/helpers/users_helper.rb>
module UsersHelper
  # 引数で与えられたユーザーのGravatar画像を返す
  def gravatar_for(user)
    gravatar_id = Digest::MD5::hexdigest(user.email.downcase) # ここでメールアドレスを小文字にしてMD5という種類のハッシュ化を行う
    gravatar_url = "https://secure.gravatar.com/avatar/#{gravatar_id}" # ハッシュ化した値をURLに挟んであげる
    image_tag(gravatar_url, alt: user.name, class: "gravatar") # URLを参照してアイコンを取得して、表示する
  end
end

```

Strong Paramaters

データを登録するとき、各フィールドのデータがきちんと入っているかを調べるもの。

データを登録する前にチェックを行うことで、不正なデータが無いかな等をチェックできる。

/controller/oo.rbのcreateメソッド

```

params.require(:モデル名).permit(:受け取るフィールド, :受け取るフィールド...)

```


/controller/users_controller.rb#create

```
def create
  @user = User.new(user_params) #下のメソッドを実行
  if @user.save
  else
    render 'new', status: :unprocessable_entity
  end
end
private
def user_params
  # nameとemailとpasswordとpassword_confirmationにデータが入っていないと次にいけないような設定
  params.require(:user).permit(:name, :email, :password, :password_confirmation)
end

end
```

sessionとcookie

データベースを使わずにデータを一時的に保存するためにセッションとクッキーがある。どちらもweb系システムではよく使われる仕組みになる

- cookie ... ブラウザに一時保存されるデータ。主にユーザーIDやパスワードなどが保存される。ブラウザ上にデータがあるので基本的に自由に登録、修正削除ができる。比較的、小容量のデータに使用される。
- session ... サーバーに一時保存されるデータ。主に持続情報やページを跨いだデータ保存に使用される。基本的に短時間のデータ保存に使用される。

補足)

サイトログイン時に、メールアドレスとパスワードが自動入力されるのはCookieが働いているから。メールアドレスがCookieのほうに保存されているから、判別することができる。

そしてsessionは、やや少しの間データを保存しておく。例えば、アンケートフォームで書いた内容が少しの間消えなかったりがある。

セッションの使い方

```
session[:データ名]
# でアクセスできる。

# データを代入するときは
session[:データ名] = 値

# データを参照するときは
session[:データ名]
# とする
```

sessionsコントローラとnewというviewを作成

form_withのmodelオプションを使うことでnewだったらcreate、editだったらupdateのメソッドにsubmitをクリックしたときは飛ばされる。

特定のurlに飛ばしたいときはurlオプションを使う

```
form_with(@model:インスタンス名, url: "post送信をするURL") # ※modelは省略可
```

とする

例えば(view/new.html.erb)でボタンを押せばcreate実行
(view/edit.html.erb)でボタンを押せばupdate実行をいままで自動で飛ばされていた

例

```
<%= form_with(url: login_path, scope: :session) do |f| %> #login_pathに飛ばすよ。ということ。sessionを使ったフォームにしますよということ。

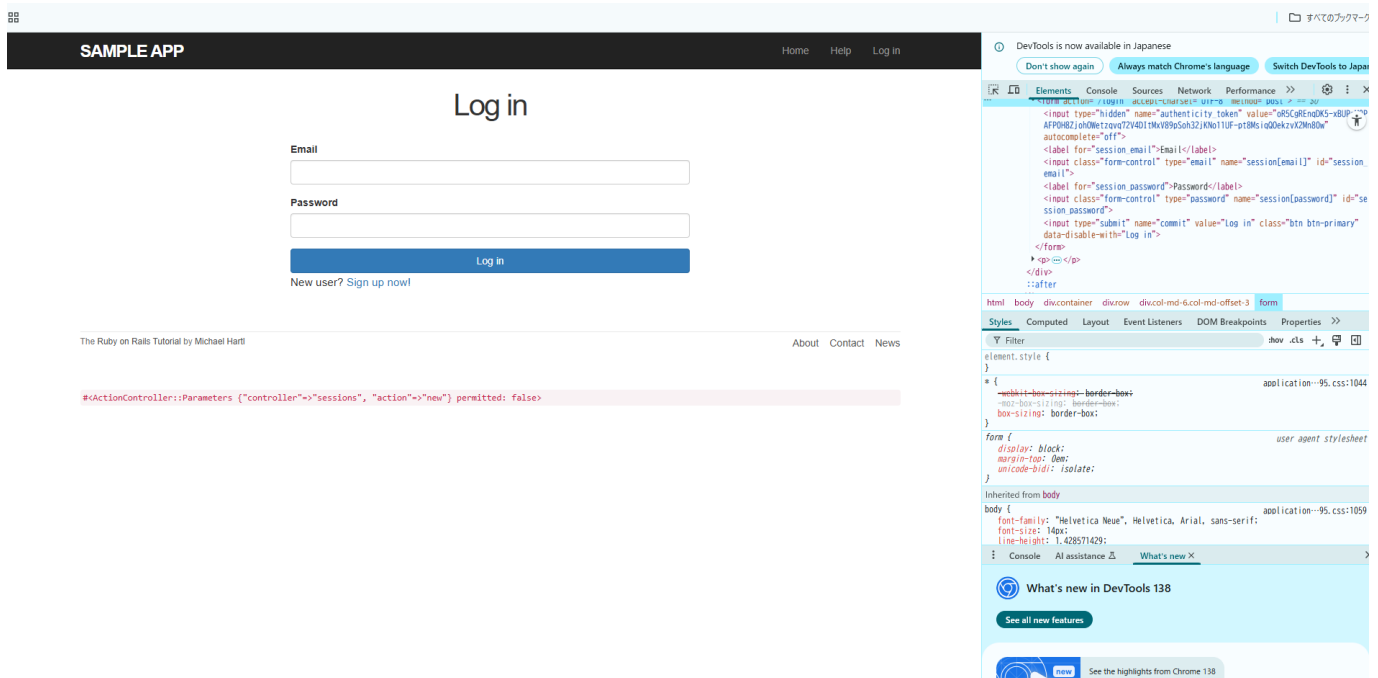
  <%= f.label :email %>
  <%= f.email_field :email, class: 'form-control' %>

  <%= f.label :password %>
  <%= f.password_field :password, class: 'form-control' %>

  <%= f.submit "Log in", class: "btn btn-primary" %>
<% end %>
```

sessionが実際のhtmlでつかわれている様子

以下はEmailとpasswordでログインする画面



htmlのformタグ内では、テキストフィールドに`session[]`と書かれている。これはこのsessionに保存する役割を果たす



formとparamsの関係

form_withの中にinputタグを使うと、htmlソースに

モデル名 [フィールド名]

という名前の入力用のフィールドが作成される

submitボタンで送信したときには、コントローラ内で

```
params[:モデル名][:フィールド名]
```

でアクセスできる

loginが二つあるとどうなるのか

route.rbに以下の設定がある

createメソッドが起動される

sessions_controller.rb

```
# createメソッド
def create
  p "-----" # ここではsubmitを押されたとき、記述されたemailなどがコンソールに表
  示
  p params[:session][:email]

  # downcaseは小文字化
  user = User.find_by(email: params[:session][:email].downcase) # ここでは

  # authenticateはパスワードを暗号化したもの
  if user && user.authenticate(params[:session][:password]) #この条件分岐は&&の左
  側のオペランドを先に照合し、nullじゃなかったら右側のオペランド式を実行する

    # ログイン

  else
    # ここではもしも上の照合で引っかからなかった（ユーザーが見つからなかった）場合に、newへ飛ばして新規登
    録させる画面
    render "new", status: :unprocessable_entity
  end
end
```

flashメッセージとは

一時的に表示したいメッセージのこと。ログインに成功した/失敗したやデータの登録の成功などを画面の上や下に表示する。

通常は一回だけ表示した後、自動的に削除される

flashメッセージ使い方

コントローラなどで

```
flash[:キー] = メッセージ
```

とする。

表示するときは

```
flash.each.do | キー用変数, メッセージ用変数 |
```

で受け取りメッセージを表示する

キーには主に

- :notice ... 成功時のメッセージ
- :alert ... 失敗したときのメッセージ

セッション固定攻撃

セッションが長い間サーバーに保存されていたら、セッション取得を使ってデータを取得できてしまう

findとfind_byの違い

- User.find(1)の場合、データが見つければそのまま続けれるが、データが無かったらエラーになってシステムが終了してしまう
- User.find_by(1)の場合、データが見つければそのまま続けるし、データが無くてもシステム終了せず、nilを返す

current_user(セッションが存在している場合のユーザーを返すメソッド)

ヘルパーとして定義してやることで、自分のユーザを返すメソッドをいろんなところで使える

```
module SessionsHelper
  def log_in(user) # ユーザを受け取り、セッションにidを格納しておく
    session[:user_id] = user.id
  end
  def current_user # セッションから現在のユーザに関する情報を取得し、返回值として返す
    @current_user ||= User.find_by(id: session[:user_id]) if session[:user_id]
  end
end
```

end

練習問題

rental_appを作成

1. 以下の条件でスキファールドを作成
モデル名: book
フィールド名: title string
price integer
2. 以下の条件でモデルを作成
モデル名: rental
フィールド名: book_id integer
haku su integer
3. rentalモデルに書きバリデーションを作成
book_id 必須チェック
haku su 必須チェック
4. 3に対するモデルテストを作成
5. コントローラrentalを作成
6. new.html.erbを作成
submitボタンを押すとcreateに飛び、rentalを登録できるようにする
7. 統合テストrentalを作成し、下記のチェックを行う
 1. rental/newに対して、下記のチェックを行う
rental/newテンプレートが使用される
 2. すべてのフィールドを空で登録すると登録できず
rental/newテンプレートを使った表示を行う

練習問題

1. プロジェクトNenpi_appを作成
rails new Nenpi_app
2. 以下の条件でスキファールドしなさい
モデル名: car
フィールド: name string型
3. 2を利用し以下の車種を登録しなさい
 - tanto
 - Move
 - カローラ
4. 以下の条件でモデルを作成しなさい
モデル名: soukou

```
フィールド: car_id integer型
           : kyori float型
```

5. コンソール画面から以下の情報を追加しなさい

```
・car_id:1
・kyori:20.5
```

```
・car_id:1
・kyori:15.5
```

6. 以下のモデルを作成しなさい

```
モデル名: kyuyu
フィールド: car_id integer
           kyuyu float
```

7. 以下のコントローラを作成しなさい

```
コントローラ名: kyuyus
```

8. 給油情報を新規入力するために仕組みを作成しなさい

※ここでroute.rbで最初は/kyuyus/new , to: "kyuyus#create"としていたが、form_with(model: @kyuyu, url: "kyuyus/create")とすることで、"kyuyus/create"というurlに情報を送ることができ、post "/kyuyus/new"だけでpost送信できるようになる

9. 下記のようなindexページを作成し、新規登録後はこのページを開きなさい

```
車種    給油
tanto   15.2
tanto   30.0
カローラ 25.5
```

10. Nenpiコントローラを作成し以下の表を作成しなさい

```
車種    走行    給油
tanto   36.0    45.2
Move    0        0
カローラ 0        25.5
```

追加

has_many:モデル名を今までしていたが、has_manyは複数形になるので、has_many:モデル名sとしないといけない